Vertical Scaling

make one machine bigger, and know when to stop

Watch the 3-min video

In one line: vertical scaling (scaling up) gives one machine more CPU, memory or faster disks: no code changes and quick relief, but it hits a ceiling, gets expensive and stays a single point of failure.

1 What it is

Scaling up means making one machine more powerful: more CPU cores, more RAM, faster storage (for example moving to NVMe SSDs), a faster network card. Your code doesn't change; the same app simply runs on stronger hardware. In the cloud it's often one setting: change the instance type and restart.

2 A worked example



Figure 1. A database at its limit, fixed by resizing the machine.

The PostgreSQL VM has 8 vCPUs and sits at 90% CPU at peak. Resizing to 32 vCPUs and more memory brings CPU to ~30%, and more of the working set now fits in memory, so queries hit the disk less. Four times the cores rarely means four times the speed (locks, single-threaded parts and I/O limit the gain), but it buys a lot of headroom fast.

3 The four limits

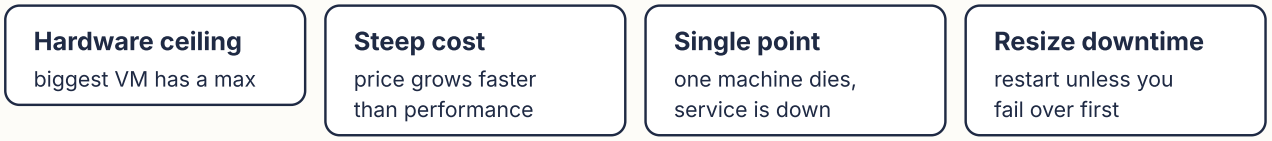


Figure 2. Why scaling up is a first step, not a final answer.

- **Hardware ceiling.** Even the largest instance has a maximum number of cores and memory. Past it, you can't go higher.
- **Steep cost curve.** The biggest and high-memory machines carry a premium, while software rarely uses every extra core well. Each extra unit of performance costs more.
- **Single point of failure.** Everything still lives on one machine. If it fails, the service is down.

- **Downtime to resize.** Changing size usually means a restart, unless you first fail over to a replica and resize the old primary.

4 When it's the right move

Situation	Why scaling up fits
You need relief today	One setting, no code or data changes
Stateful, hard-to-split workloads	A relational database with joins and transactions works best on one node
You're still small	One big machine is simpler than a cluster to run
Buying time	Gives you months to plan replicas, caching or sharding properly

SCALE UP VS SCALE OUT

Scale up first for stateful parts and early-stage systems; scale out for stateless tiers and when you need fault tolerance or growth beyond one machine. Most real systems do both: a big primary database plus many small app servers.

COMMON MISTAKES

- Dismissing vertical scaling as “not scalable”: it's often the cheapest first fix.
- Scaling up forever and discovering the ceiling during a traffic spike.
- Forgetting it doesn't improve availability.
- Resizing the primary without a replica to fail over to.

5 Key terms

Vertical scaling: a bigger machine (scale up)

vCPU: a virtual CPU core in the cloud

Failover: switching to a standby

Horizontal scaling: more machines (scale out)

Working set: data accessed often; best kept in RAM

Headroom: spare capacity for spikes

6 Interview questions

Q1. When would you scale vertically instead of horizontally? BASIC

Early on, for stateful systems like a relational database, and when you need a quick fix without code changes. It buys time to plan horizontal scaling properly.

Q2. What are the limits of vertical scaling? BASIC

A hardware ceiling, a steep cost curve, a single point of failure, and downtime during resize unless you fail over first.

Q3. Does vertical scaling improve availability? BASIC

No. It's still one machine, so one failure takes the service down. Availability needs redundancy: a replica and failover.

Q4. Your database is at 90% CPU at peak. What do you do this week? SCENARIO

Check for slow queries and missing indexes first (often the cheapest fix), then resize the instance via a replica failover. In parallel, plan read replicas or caching for growth.

Q5. How do you resize a production database with minimal downtime? SCENARIO

Create or use a replica, resize it, promote it to primary (a few seconds of failover), then resize or replace the old primary as the new replica.

Q6. Doubling the cores only gave 30% more throughput. Why? SCENARIO

Something else is the bottleneck: lock contention, a single-threaded part, disk I/O or the network. Amdahl's Law: the serial part limits the gain from more parallel hardware.

Q7. Is “just get a bigger machine” ever the best senior answer? TRICKY

Yes, when it's the fastest, cheapest way to meet the requirement and the growth forecast fits under the ceiling. Say so, and name the trigger for moving to horizontal scaling.

Q8. Why are high-memory instances so expensive per GB? TRICKY

They're specialised hardware made in smaller volumes, and cloud pricing at the top end includes a premium. Performance per dollar falls as size grows.

Next: SD-A-007 · Horizontal Scaling backendroadmap.com